



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Друзья, всех приветствую! В этом уроке мы разберем работу с условными операторами. Очень важно изучить предыдущий модуль, посвященный написанию Bash сценариев так как эти знания обязательно будут нужны (полезны) при реальном проведении тестирования на проникновение.

Условные операторы (операторы ветвления) позволяют нам выполнять различные действия на основе различных условий.

Ветвление в Bash осуществляют операторы if, else и elif.

Оператор **if** относительно прост. Он проверяет, истинно ли условие, и имеет очень специфический синтаксис. Обратите пристальное внимание на этот синтаксис, особенно на использование обязательных пробелов:

```
if [ <УСЛОВИЕ> ]  
then  
  <ДЕЙСТВИЕ>  
fi
```

Оператор then можно расположить на той же строке, что и условие, но в этом случае перед ним нужно поставить точку с запятой

```
if [ <УСЛОВИЕ> ] ; then  
  <ДЕЙСТВИЕ>  
fi
```

Давайте рассмотрим реальный пример:

```
kali@kali:~$ cat > ./if.sh  
#!/bin/bash  
read -p "Ваш возраст: "age  
if [ $age -lt 18 ] ; then  
  echo "Вам может понадобиться разрешение родителей, чтобы пройти этот курс! :)"  
fi  
kali@kali:~$ chmod +x ./if.sh  
kali@kali:~$ ./if.sh  
Ваш возраст: 15  
Вам может понадобиться разрешение родителей, чтобы пройти этот курс! :)
```

В этом примере мы использовали оператор **if** для проверки возраста, введенного пользователем. Если введенный возраст меньше (**-lt**) 18, сценарий выводит предупреждающее сообщение. Квадратные скобки ("**[**" и "**]**") в операторе **if** на самом деле являются ссылкой на команду **test**. Это просто означает, что мы можем использовать все операторы, которые разрешены командой **test**. Некоторые из наиболее распространенных операторов включают:

Оператор	Описание: Выражение истинно, если...
!EXPRESSION	Выражение EXPRESSION ложно.
-n STRING	Длина строки STRING больше нуля.
-z STRING	Длина строки STRING равна нулю (пустая строка).
STRING1 != STRING2	STRING1 не равен STRING2.
STRING1 = STRING2	STRING1 равен STRING2.
INTEGER1 -eq INTEGER2	INTEGER1 равен INTEGER2.
INTEGER1 -ne INTEGER2	INTEGER1 не равен INTEGER2.
INTEGER1 -gt INTEGER2	INTEGER1 больше, чем INTEGER2.
INTEGER1 -lt INTEGER2	INTEGER1 меньше, чем INTEGER2.
INTEGER1 -ge INTEGER2	INTEGER1 больше или равен INTEGER2.
INTEGER1 -le INTEGER2	INTEGER1 меньше или равен INTEGER2.
-d FILE	Файл FILE существует и является каталогом.
-e FILE	Файл FILE существует.
-r FILE	Файл FILE существует и имеет право на чтение.
-s FILE	Файл FILE существует и не пуст.
-w FILE	Файл FILE существует и имеет право на запись.
-x FILE	Файл FILE существует и имеет право на выполнение.

Соответственно, наш предыдущий пример с использованием **if** можно переписать без квадратных скобок следующим образом:

```
kali@kali:~$ cat > ./if2.sh
#!/bin/bash
read -p "Ваш возраст: "age
if test $age -lt 18
then
echo "Вам может понадобится разрешение родителей, чтобы пройти этот курс! :)"
fi
kali@kali:~$ chmod +x ./if2.sh
kali@kali:~$ ./if2.sh
Ваш возраст: 15
Вам может понадобится разрешение родителей, чтобы пройти этот курс! :)
```

Несмотря на то, что этот пример функционально эквивалентен примеру с использованием квадратных скобок, использование квадратных скобок делает код немного легче для чтения. Мы также можем выполнить определенный набор действий, если утверждение истинно, и другой набор, если оно ложно. Для этого мы можем использовать оператор **else**, который имеет следующий синтаксис:

```
if [ <УСЛОВИЕ> ]
then
<ДЕЙСТВИЕ>
else
<ДРУГОЕ ДЕЙСТВИЕ>
fi
```

Давайте расширим наш предыдущий пример, включив в него оператор **else**:

```
kali@kali:~$ cat > ./else.sh
#!/bin/bash
read -p "Ваш возраст: "age
if [ $age -lt 18 ]
```

```
then
echo "Вам может понадобится разрешение родителей, чтобы пройти этот
курс! :)"
else
echo "Добро пожаловать на курс!"
fi
kali@kali:~$ chmod +x ./else.sh
kali@kali:~$ ./else.sh
Ваш возраст: 21
Добро пожаловать на курс!
```

Обратите внимание, что оператор `else` выполнялся, когда введенный возраст был больше (или, точнее, не меньше) восемнадцати лет. Операторы **`if`** и **`else`** допускают только две ветви выполнения кода. Мы можем добавить дополнительные ветви с помощью оператора **`elif`**, который использует следующую схему:

```
if [ <УСЛОВИЕ> ]
then
<ДЕЙСТВИЕ>
elif [ <ДРУГОЕ УСЛОВИЕ> ]
then
<РАЗЛИЧНЫЕ ДЕЙСТВИЯ>
else
<ДРУГИЕ РАЗЛИЧНЫЕ ДЕЙСТВИЯ>
fi
```

Давайте снова расширим наш пример, включив в него оператор **`elif`**:

```
kali@kali:~$ cat > ./elif.sh
#!/bin/bash
read -p "Ваш возраст: "age
if [ $age -lt 18 ]
then
```



```
echo "Вам может понадобится разрешение родителей, чтобы пройти этот курс! :)"
elif [ $age -gt 60 ]
# -gt - больше
then
echo "Снимаю перед вами шляпу!"
else
echo "Добро пожаловать на курс!"
fi
kali@kali:~$ chmod +x ./elif.sh
kali@kali:~$ ./elif.sh
Ваш возраст: 65
Снимаю перед вами шляпу!
```

В этом примере синтаксис кода был немного сложнее. В порядке выполнения ветвь **then** выполняется, если введенный возраст меньше восемнадцати лет, ветвь **elif** выполняется если возраст больше шестидесяти, а ветвь **else** выполняется только в том случае, если возраст больше восемнадцати, но меньше шестидесяти.

Использование условных операторов в Bash является ключевым элементом в программировании, так как они позволяют создавать сценарии, которые адаптируются к различным условиям и данным, с которыми они работают. Эти приемы помогают автоматизировать задачи, делая программы более интерактивными и способными принимать решения на основе входных данных, что значительно увеличивает их гибкость и универсальность.

Например, вы можете написать скрипт, который выполняет разные действия в зависимости от того, какие файлы присутствуют в системе, какой пользователь запускает скрипт или какие параметры были переданы. Это делает ваши программы не только более мощными, но и позволяет эффективно решать широкие задачи, от простых проверок до сложной логики принятия решений.